

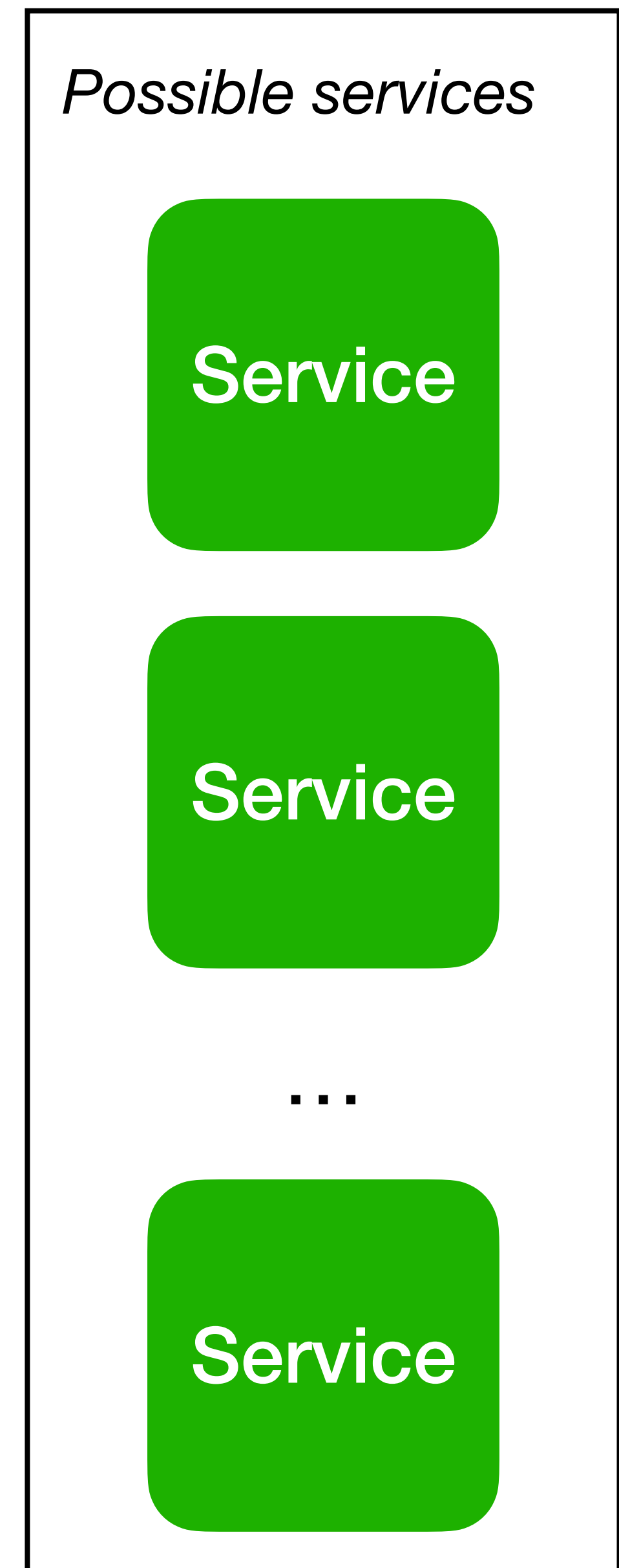
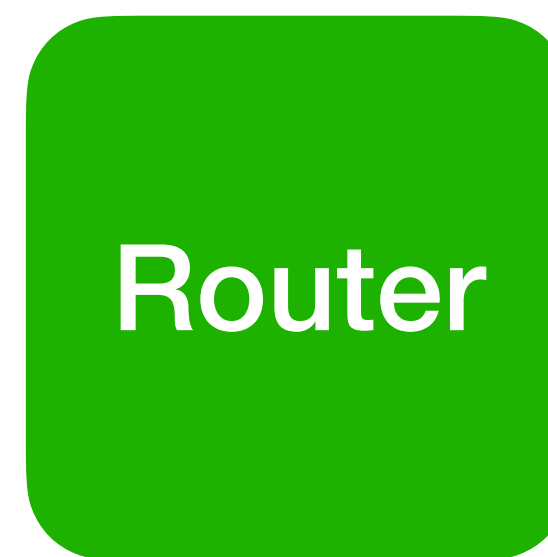
Blame-Correct Support for Receiver Properties in Recursively-Structured Actor Contracts

[Bram Vandenbogaerde](#), Quentin Stiévenart, Coen De Roover



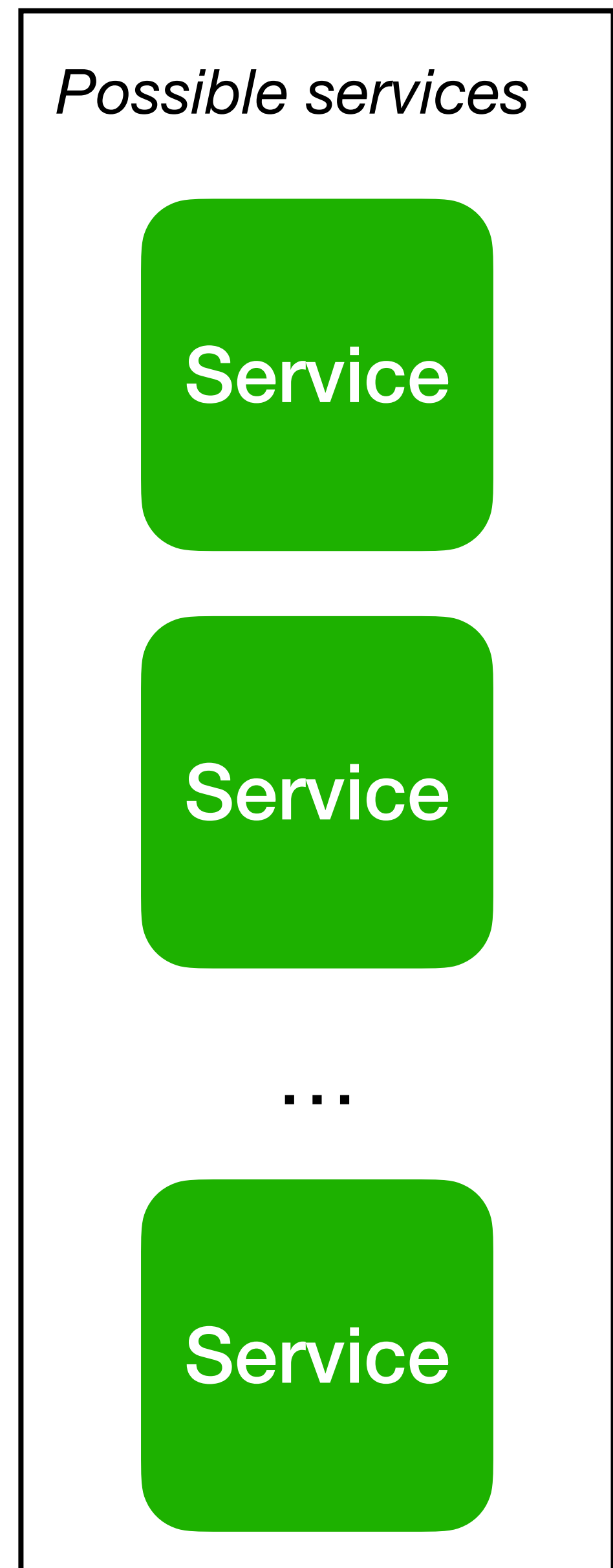
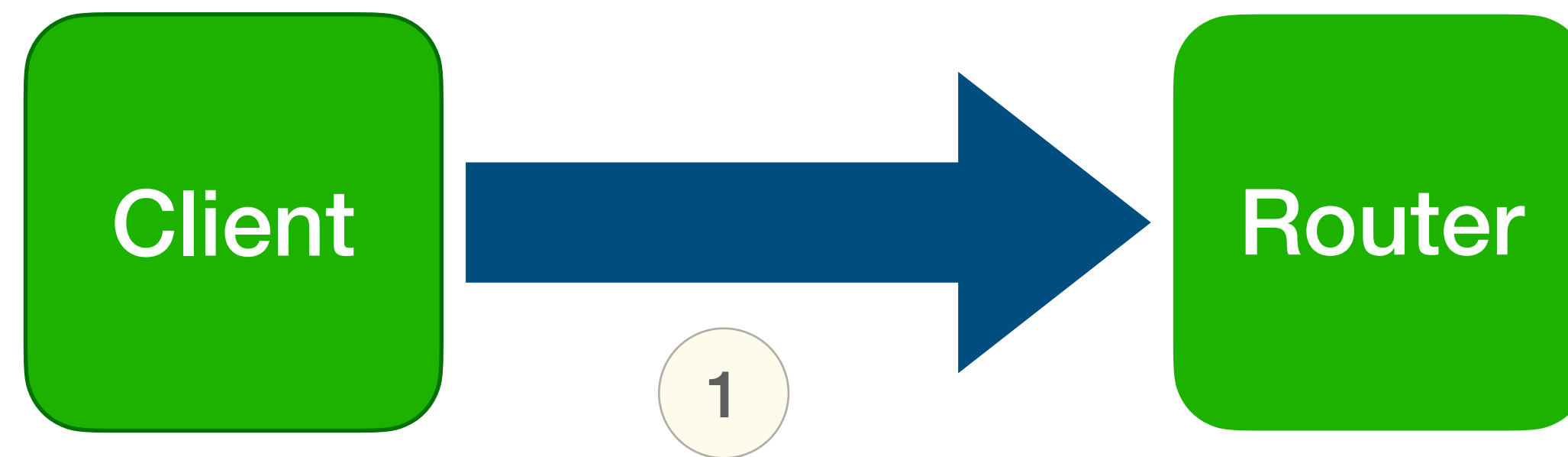
Motivating Example

Forward Flow in a Distributed System



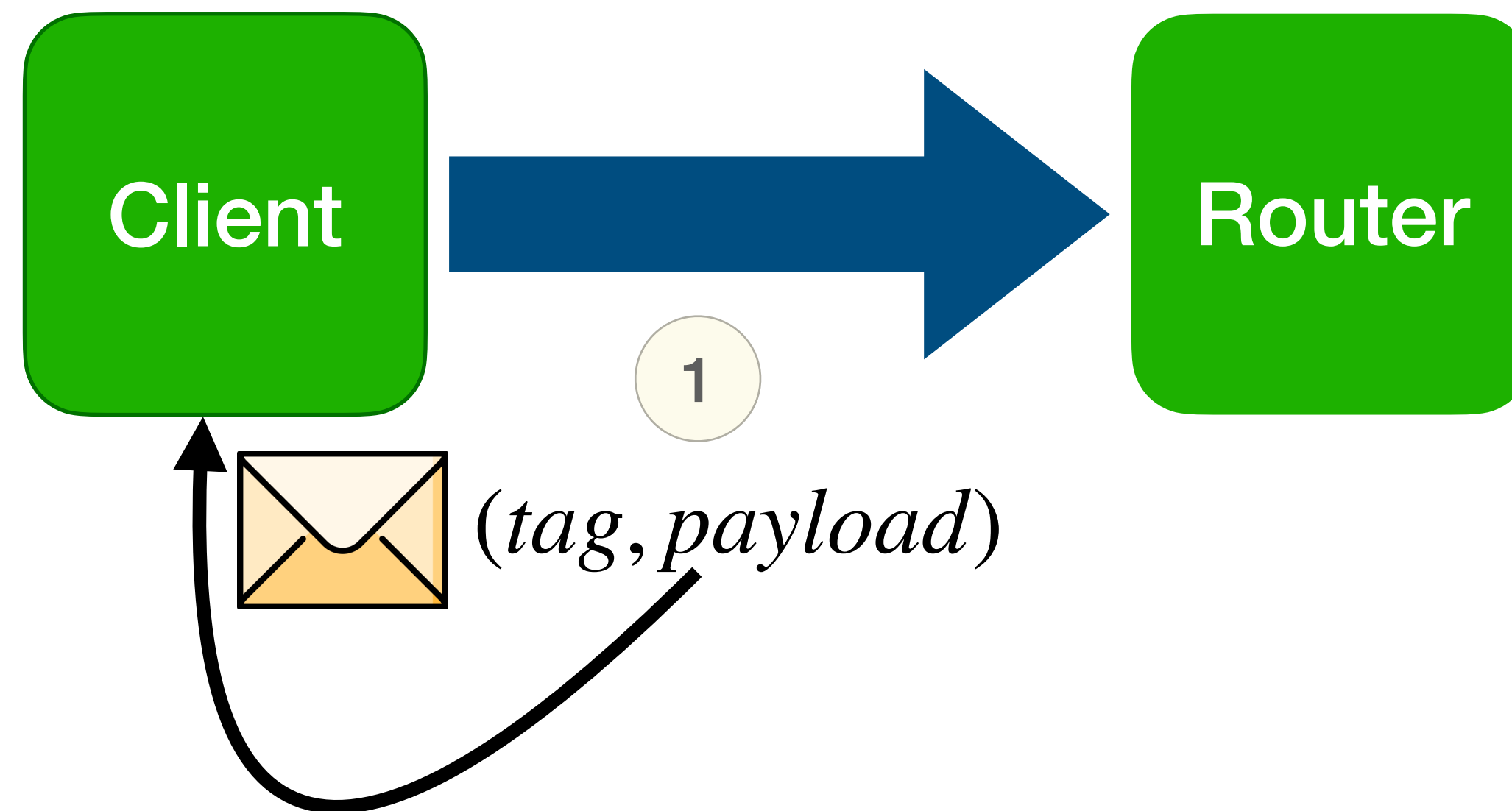
Motivating Example

Forward Flow in a Distributed System



Motivating Example

Forward Flow in a Distributed System



Possible services

Service

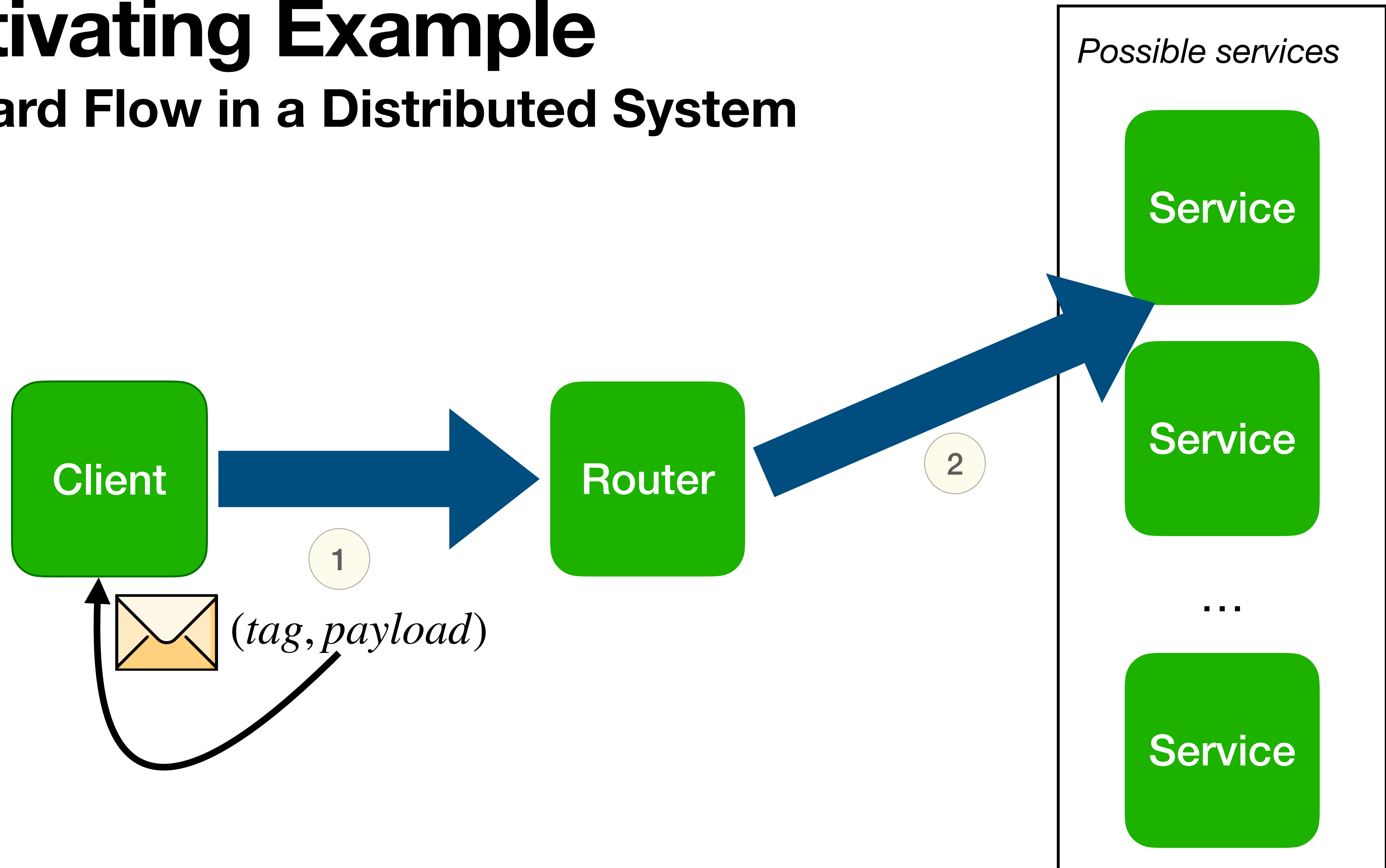
Service

...

Service

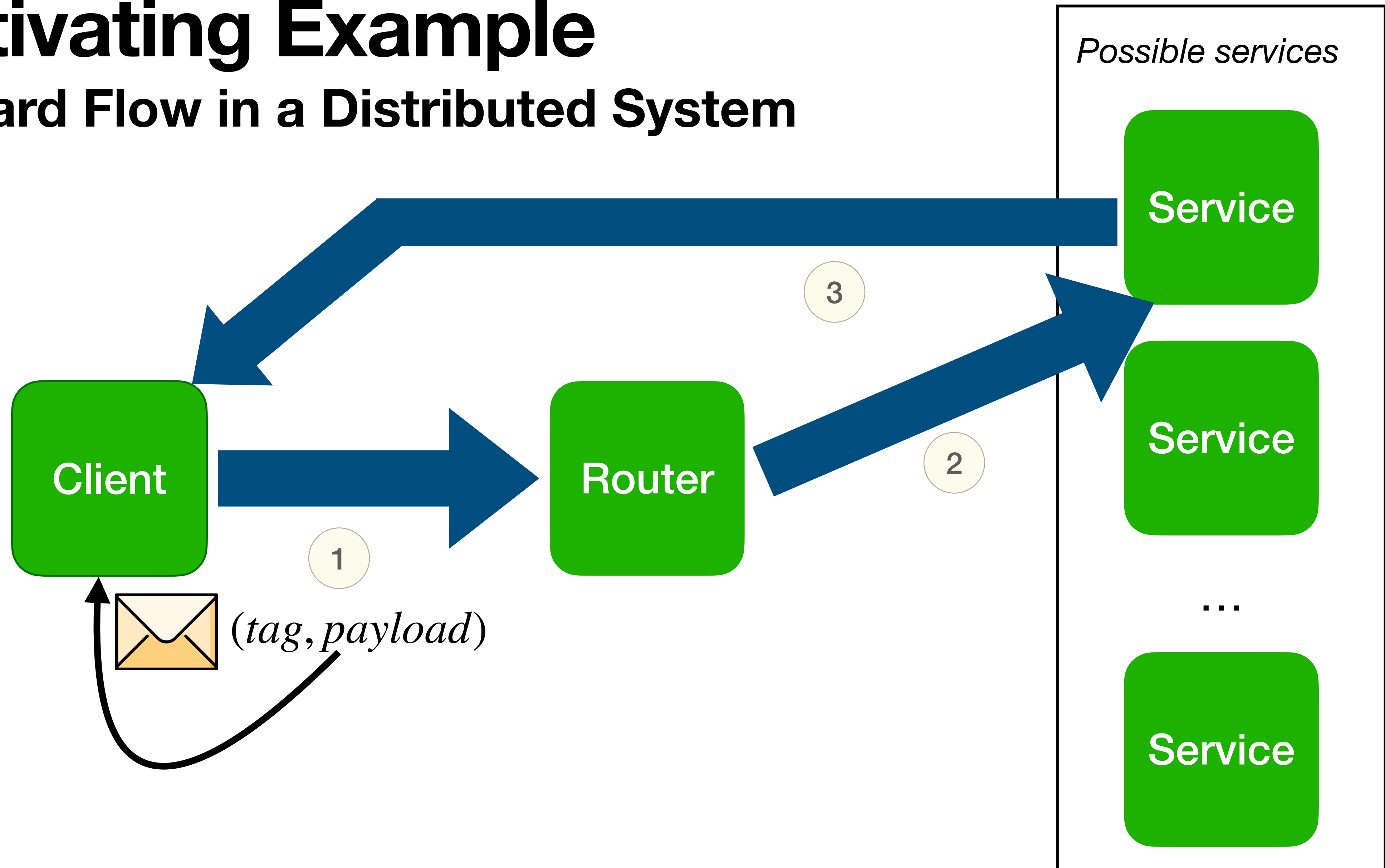
Motivating Example

Forward Flow in a Distributed System



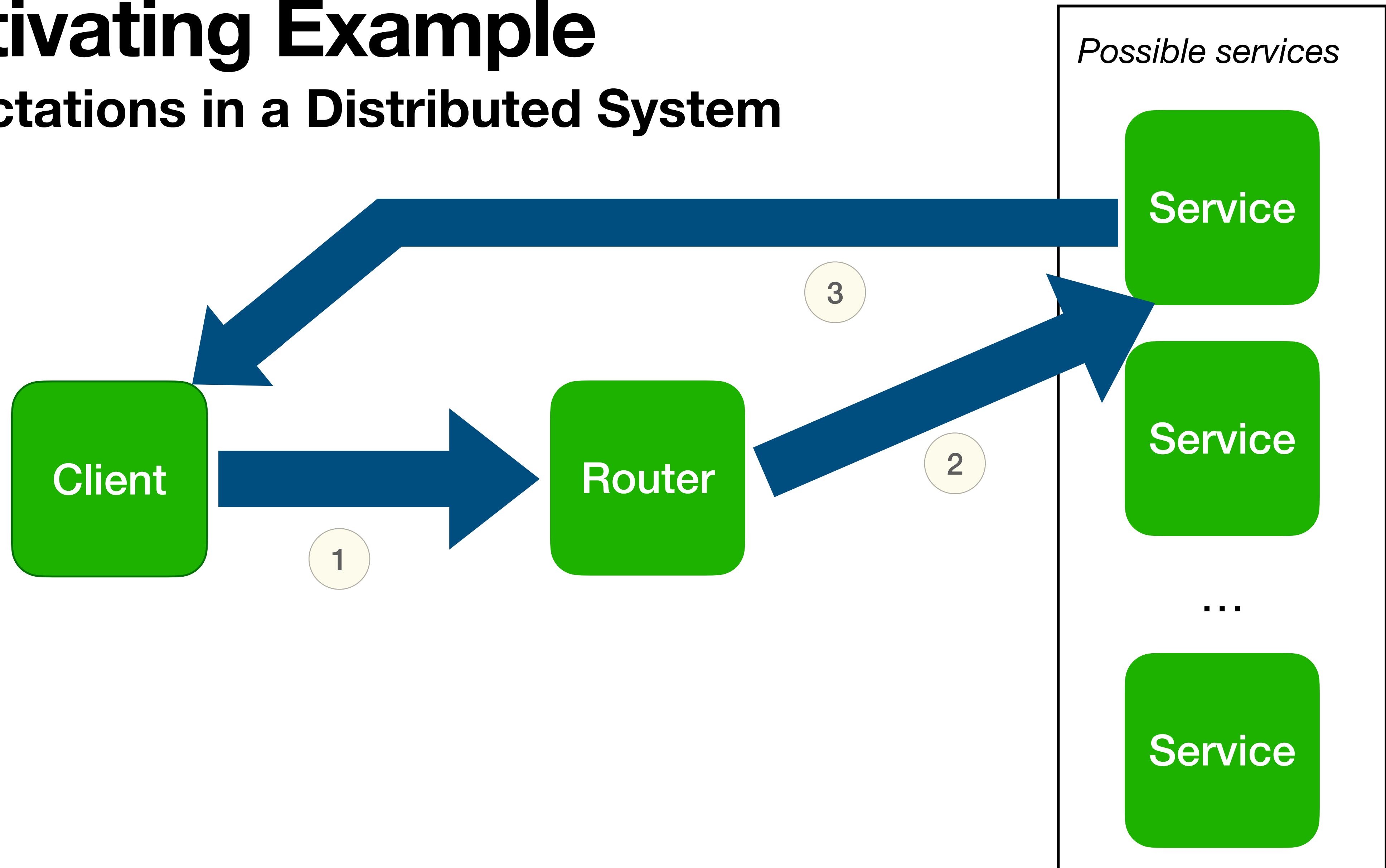
Motivating Example

Forward Flow in a Distributed System



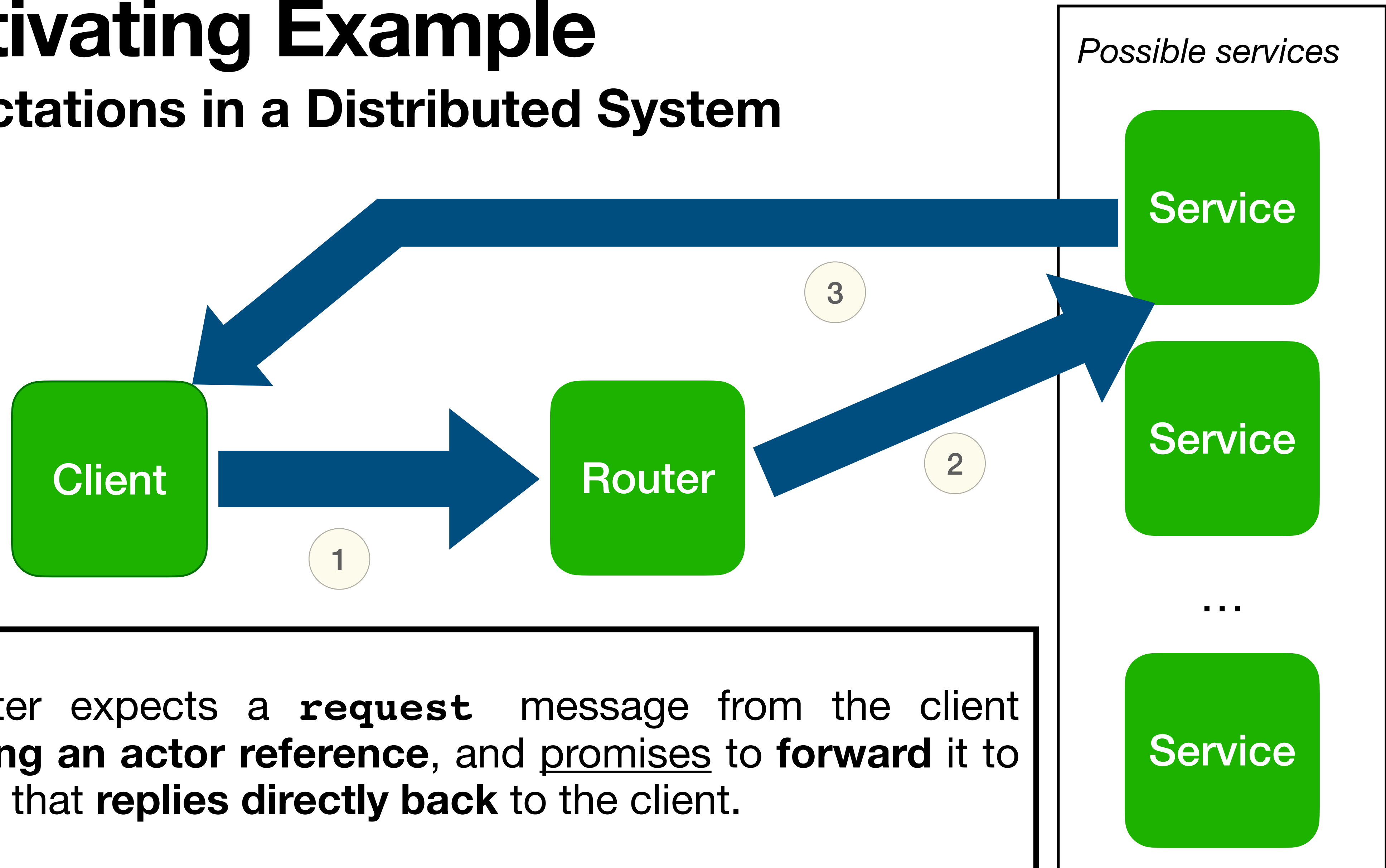
Motivating Example

Expectations in a Distributed System



Motivating Example

Expectations in a Distributed System



Expectations as Contract Specifications

The router expects a **request** message from the client **containing an actor reference**, and promises to **forward** it to a service that **replies directly back** to the client.

Expectations as Contract Specifications

The router expects a **request** message from the client **containing an actor reference**, and promises to **forward** it to a service that **replies directly back** to the client.

Expected actor interface

The router expects a **request** message from the client

Expectations as Contract Specifications

The router expects a **request** message from the client **containing an actor reference**, and promises to **forward** it to a service that **replies directly back** to the client.

Expected actor interface

The router expects a **request** message from the client

Expected payload

Payload should contain actor reference

Expectations as Contract Specifications

The router expects a **request** message from the client **containing an actor reference**, and promises to **forward** it to a service that **replies directly back** to the client.

Expected actor interface

The router expects a **request** message from the client

Expected payload

Payload should contain actor reference

Expected communication behavior

Router forwards message to the service and service replies to the client

Expectations as Contract Specifications

The router expects a **request** message from the client **containing an actor reference**, and promises to **forward** it to a service that **replies directly back** to the client.

Expected actor interface

The router expects a **request** message from the client

Expected payload

Payload should contain actor reference

Expected communication behavior

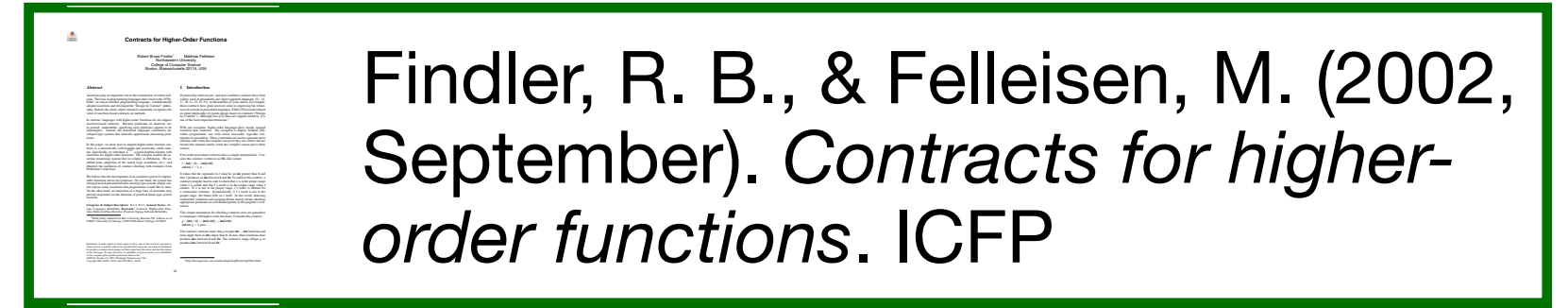
Router forwards message to the service and service replies to the client



How can we make these
expectations explicit?

Design-by-Contract

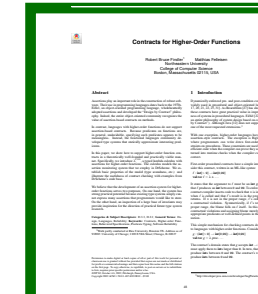
Higher-Order Sequential Contracts



Findler, R. B., & Felleisen, M. (2002, September). *Contracts for higher-order functions*. ICFP

Design-by-Contract

Higher-Order Sequential Contracts



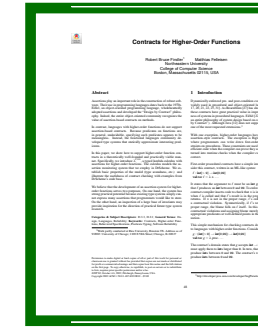
Findler, R. B., & Felleisen, M. (2002, September). *Contracts for higher-order functions*. ICFP

math.rkt

```
(define/contract (factorial x)
  (-> (>=/c 0) (>=/c 0))
  (if (= x 0)
      1
      (* x (factorial (- x 1)))))
```

Design-by-Contract

Higher-Order Sequential Contracts



Findler, R. B., & Felleisen, M. (2002, September). *Contracts for higher-order functions*. ICFP

math.rkt

```
(define/contract (factorial x)
  (-> (>=/c 0) (>=/c 0))
  (if (= x 0)
      1
      (* x (factorial (- x 1)))))
```

user.rkt

```
(require "./math.rkt")

(factorial -1)
```

Design-by-Contract

Higher-Order Sequential Contracts

 Findler, R. B., & Felleisen, M. (2002, September). *Contracts for higher-order functions*. ICFP

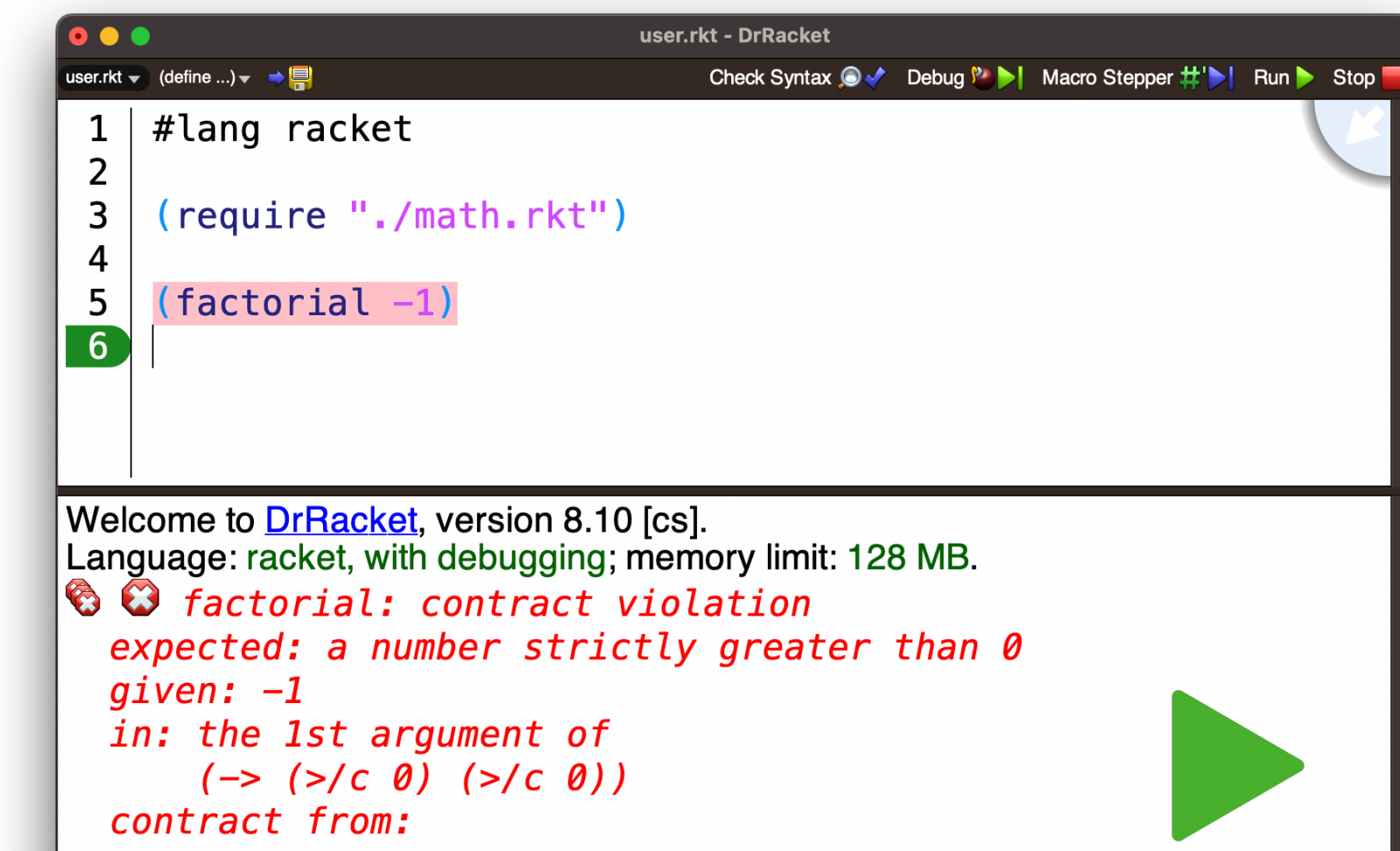
math.rkt

```
(define/contract (factorial x)
  (-> (>=/c 0) (>=/c 0))
  (if (= x 0)
      1
      (* x (factorial (- x 1)))))
```

user.rkt

```
(require "./math.rkt")

(factorial -1)
```



Design-by-Contract

Higher-Order Sequential Contracts

 Findler, R. B., & Felleisen, M. (2002, September). *Contracts for higher-order functions*. ICFP

math.rkt

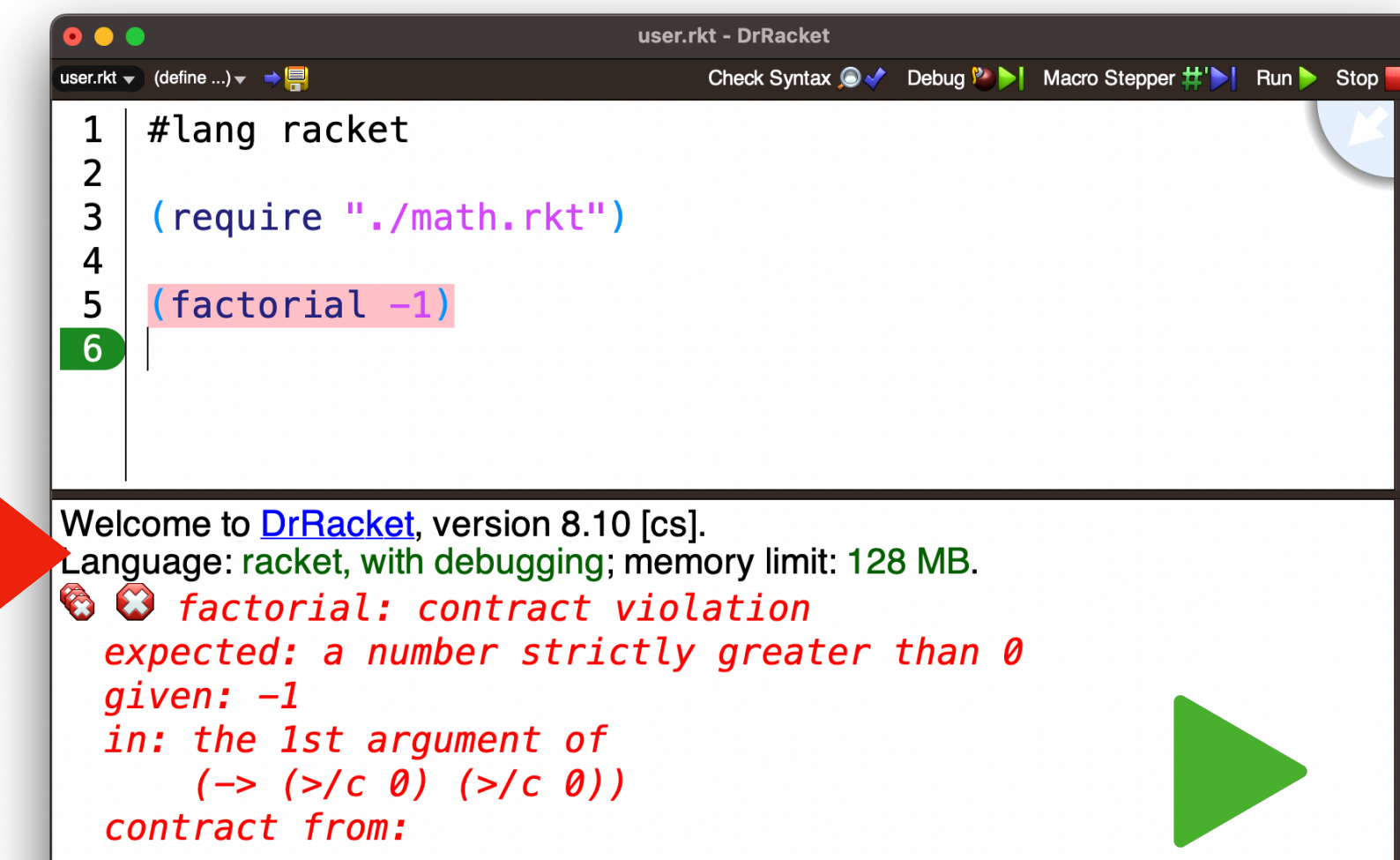
```
(define/contract (factorial x)
  (-> (>=/c 0) (>=/c 0))
  (if (= x 0)
      1
      (* x (factorial (- x 1)))))
```

Contract violations include information **blaming** the party responsible for the contract violation

user.rkt

```
(require "./math.rkt")

(factorial -1)
```



```
1 #lang racket
2
3 (require "./math.rkt")
4
5 (factorial -1)
6
```

Welcome to [DrRacket](#), version 8.10 [cs].
Language: racket, with debugging; memory limit: 128 MB.
✖ factorial: contract violation
expected: a number strictly greater than 0
given: -1
in: the 1st argument of
(-> (>=/c 0) (>=/c 0))
contract from:

Design-by-Contract

Higher-Order Sequential Contracts

Findler, R. B., & Felleisen, M. (2002, September). *Contracts for higher-order functions*. ICFP

math.rkt

```
(define/contract (factorial x)
  (-> (>=/c 0) (>=/c 0))
  (if (= x 0)
      1
      (* x (factorial (- x 1)))))
```

Contract violations include information **blaming** the party responsible for the contract violation

user.rkt

```
(require "./math.rkt")

(factorial -1)
```



The screenshot shows the DrRacket IDE with the 'user.rkt' file open. The code in the editor is:

```
1 #lang racket
2
3 (require "./math.rkt")
4
5 (factorial -1)
6
```

The output window at the bottom displays the following error message:

```
Welcome to DrRacket, version 8.10 [cs].
Language: racket, with debugging; memory limit: 128 MB.
factorial: contract violation
expected: a number strictly greater than 0
given: -1
in: the 1st argument of
      (-> (>=/c 0) (>=/c 0))
contract from:
```

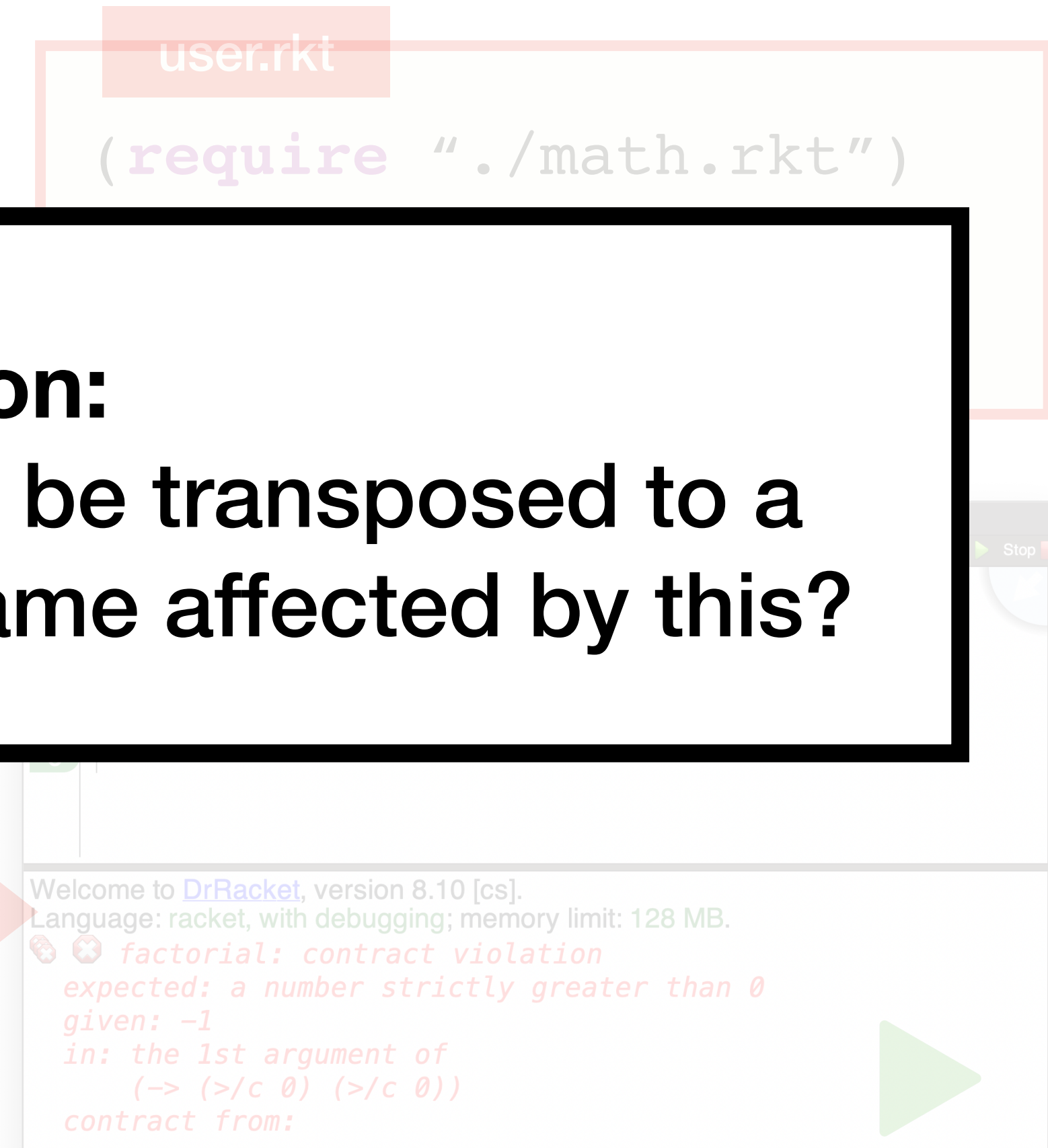
A green play button icon is visible in the bottom right corner of the output window.

Design-by-contract

Higher-Order Sequential Contracts

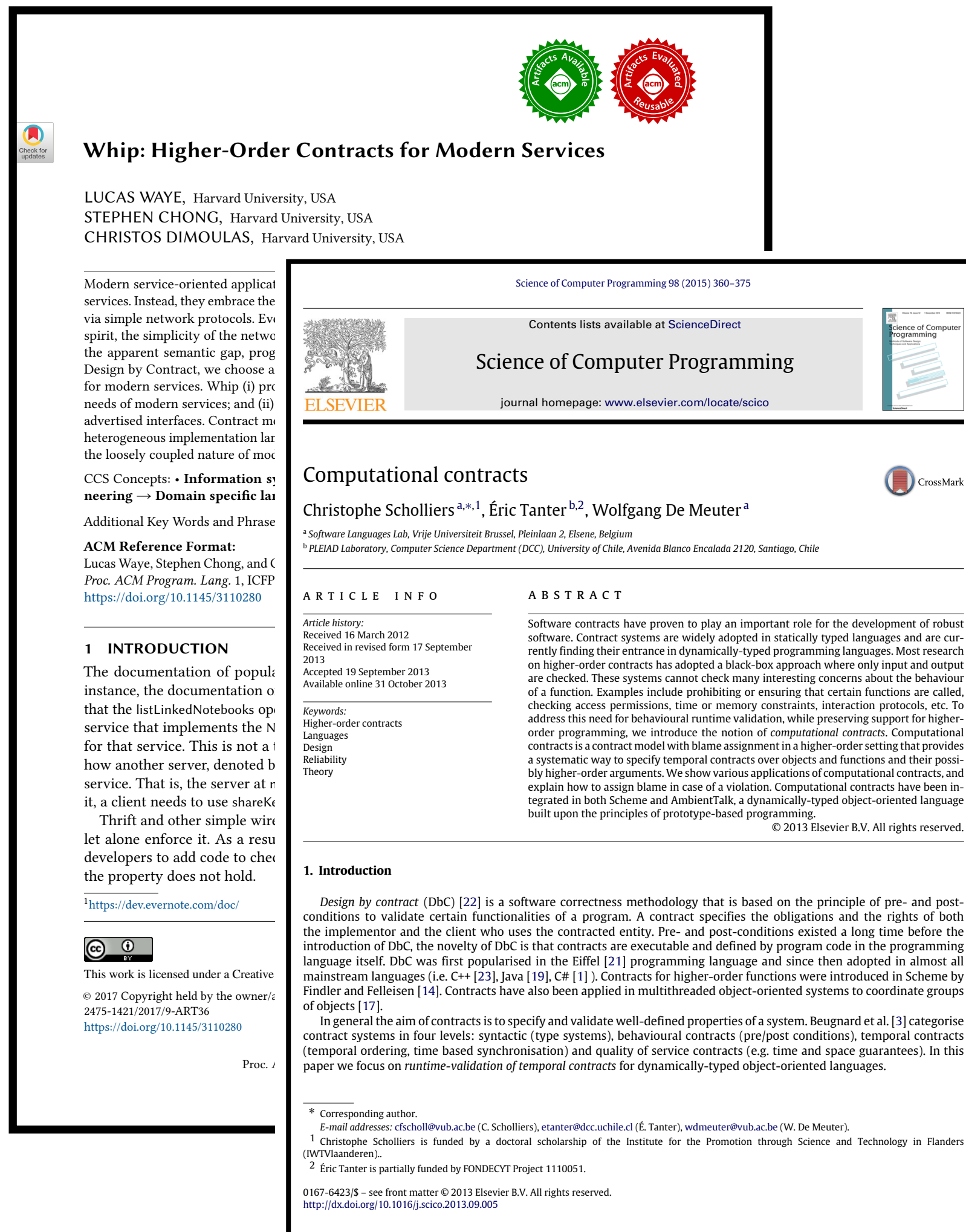
Central question:
How can Findler-style contracts be transposed to a distributed setting and how is blame affected by this?

Contract violations result in errors blaming the party responsible for the contract violation



Related Work

Other Findler-style contracts for distributed systems



No communication chains

Only two components (i.e., client - server)

No constraints on the receiver of messages

Expectations as Contract Specifications

Expected actor interface

The router expects a **request** message from the client

Expected payload

Payload should contain actor reference

Expected communication behavior

Router forwards message to the service and service replies to the client

Expectations as Contract Specifications

Expected actor interface

The router expects a **request** message from the client

Actor interface contracts

Expected payload

Payload should contain actor reference

Expected communication behavior

Router forwards message to the service and service replies to the client

Expectations as Contract Specifications

Expected actor interface

The router expects a **request** message from the client

Actor interface contracts

Expected payload

Payload should contain actor reference

Message contracts

Expected communication behavior

Router forwards message to the service and service replies to the client

Expectations as Contract Specifications

Expected actor interface

The router expects a **request** message from the client

Actor interface contracts

Expected payload

Payload should contain actor reference

Message contracts

Expected communication behavior

Router forwards message to the service and service replies to the client

Communication contracts

Receiver contracts

Expectations as Contract Specifications

Expected actor interface

The router expects a **request** message from the client

Actor interface contracts

Expected payload

Payload should contain actor reference

Message contracts

Expected communication behavior

Router forwards message to the service and service replies to the client

Communication contracts

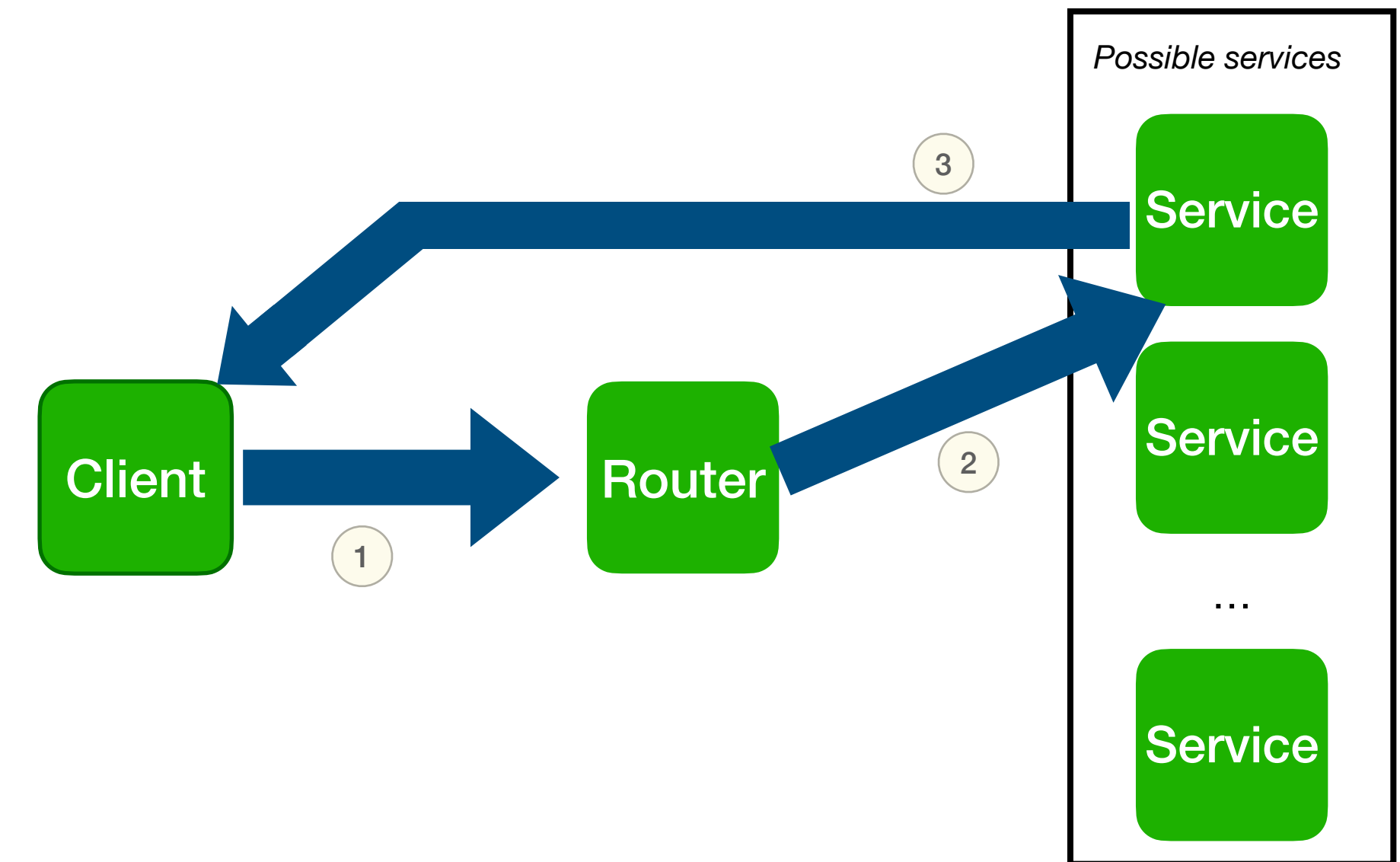
Receiver contracts

Contract for the Router

Expected actor interface

Expected payload

Expected communication



```
(message/c request-tag  
  (list actor?)  
  (lambda (payload)  
    (ensures/c  
      (list (message/c request-tag  
        (list actor?)  
        any-recipient  
        (ensures/c  
          (list (message/c reply  
            reply-contracts  
              (specific-recipient (first payload))  
              unconstrained/c)))))))))
```

Message Contracts

1

```
(message/c expected-tag  
  payload-contract  
  recipient-contract  
  communication-contract)
```

Message Contracts

1

Expected Payload

```
(message/c expected-tag  
  payload-contract  
  recipient-contract  
  communication-contract)
```

Expectation: payload should contain actor reference

```
(message/c request  
  (list actor?)  
  recipient-contract  
  communication-contract)
```

Receiver Contracts

2

Expected Receiver

```
(message/c expected-tag  
  payload-contract  
  recipient-contract  
  communication-contract)
```

Receiver Contracts

2

Expected Receiver

```
(message/c expected-tag  
  payload-contract  
  recipient-contract  
  communication-contract)
```

Expectation: router should forward the request **to a service**

```
(message/c request  
  (list actor?)  
  (lambda (payload)  
    (specific-recipient service))  
  communication-contract)
```

Contracts on Communication Effects

3

What Messages should a Receiver Send?

```
(message/c expected-tag  
  payload-contract  
  recipient-contract  
  communication-contract)
```

```
(ensures/c  
  message-contract-1  
  message-contract-2  
  ...)
```

Contracts on Communication Effects

3

What Messages should a Receiver Send?

```
(message/c expected-tag  
  payload-contract  
  recipient-contract  
  communication-contract)
```

```
(ensures/c  
  message-contract-1  
  message-contract-2  
  ...)
```



Communication and message contracts form a **mutually recursive structure** for **expressing communication chains**.

Contracts on Communication Effects

3

What Messages should a Receiver Send?

Contracts on Communication Effects

3

What Messages should a Receiver Send?

Expectation: service should directly reply to the client

Contracts on Communication Effects

3

What Messages should a Receiver Send?

Expectation: service should directly reply to the client

```
(lambda (payload)
  (ensures/c
    (list (message/c request
                     (cons actor? request-contracts)
                     (lambda (payload)
                       (specific-recipient service)
                       (ensures/c
                        (message/c reply reply-contracts
                                   (specific-recipient (first payload))
                                   unconstrained/c))))))
```

Contracts on Communication Effects

3

What Messages should a Receiver Send?

Expectation: service should directly reply to the client

```
(lambda (payload)
  (ensures/c
    (list (message/c request
      (cons actor? request-contracts)
      (lambda (payload)
        (specific-recipient service)
        (ensures/c
          (message/c reply reply-contracts
            (specific-recipient (first payload))
            unconstrained/c))))))
```



Contracts can depend on **the payload of earlier messages**

Summary

Expected actor interface

The router expects a **request** message from the client

Actor interface contracts

Key contributions:

Contracts can express communication chains that can depend on earlier communication behavior and can encode expected receiver

cts

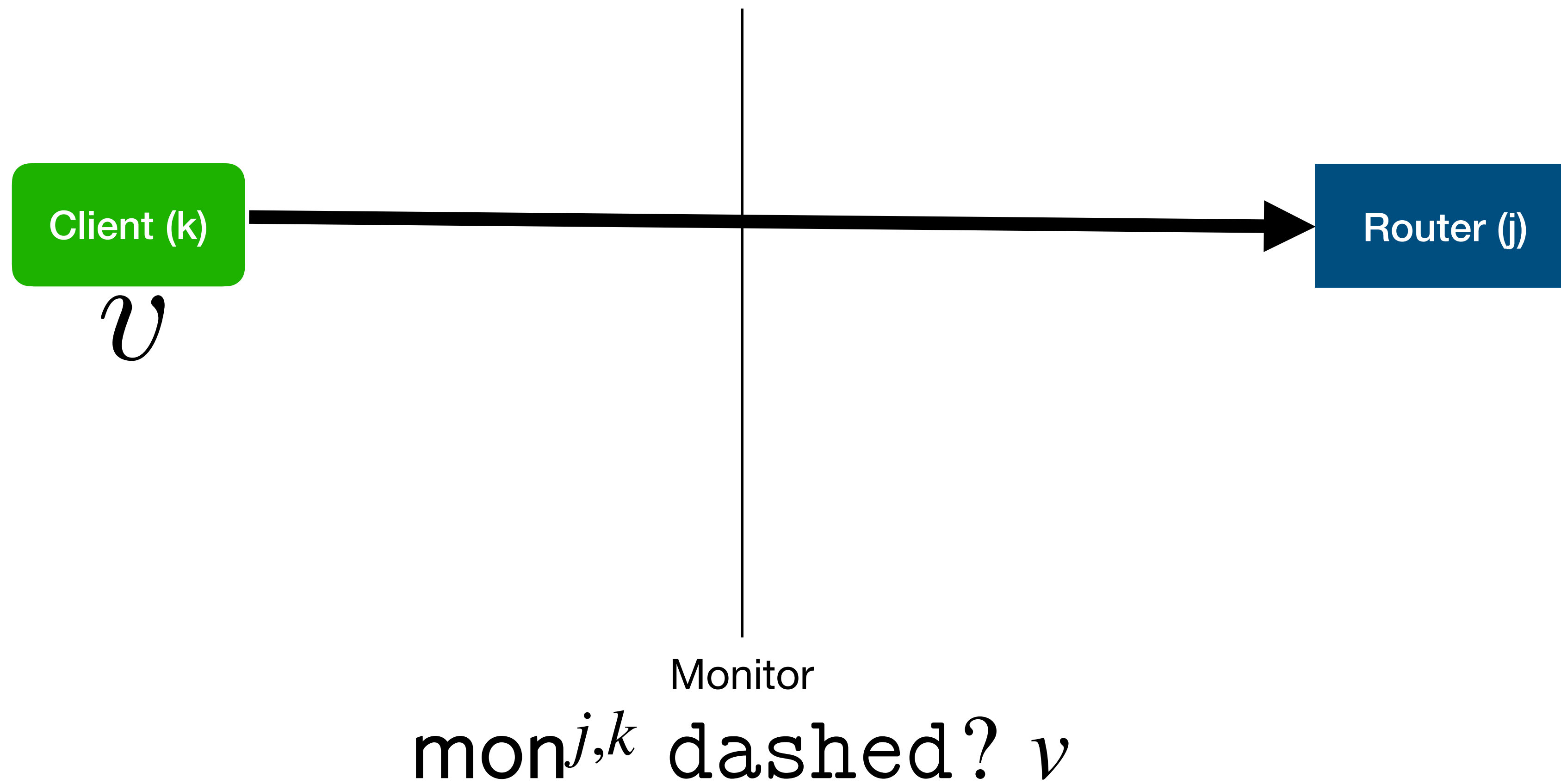
acts

Receiver contracts

Blame Assignment

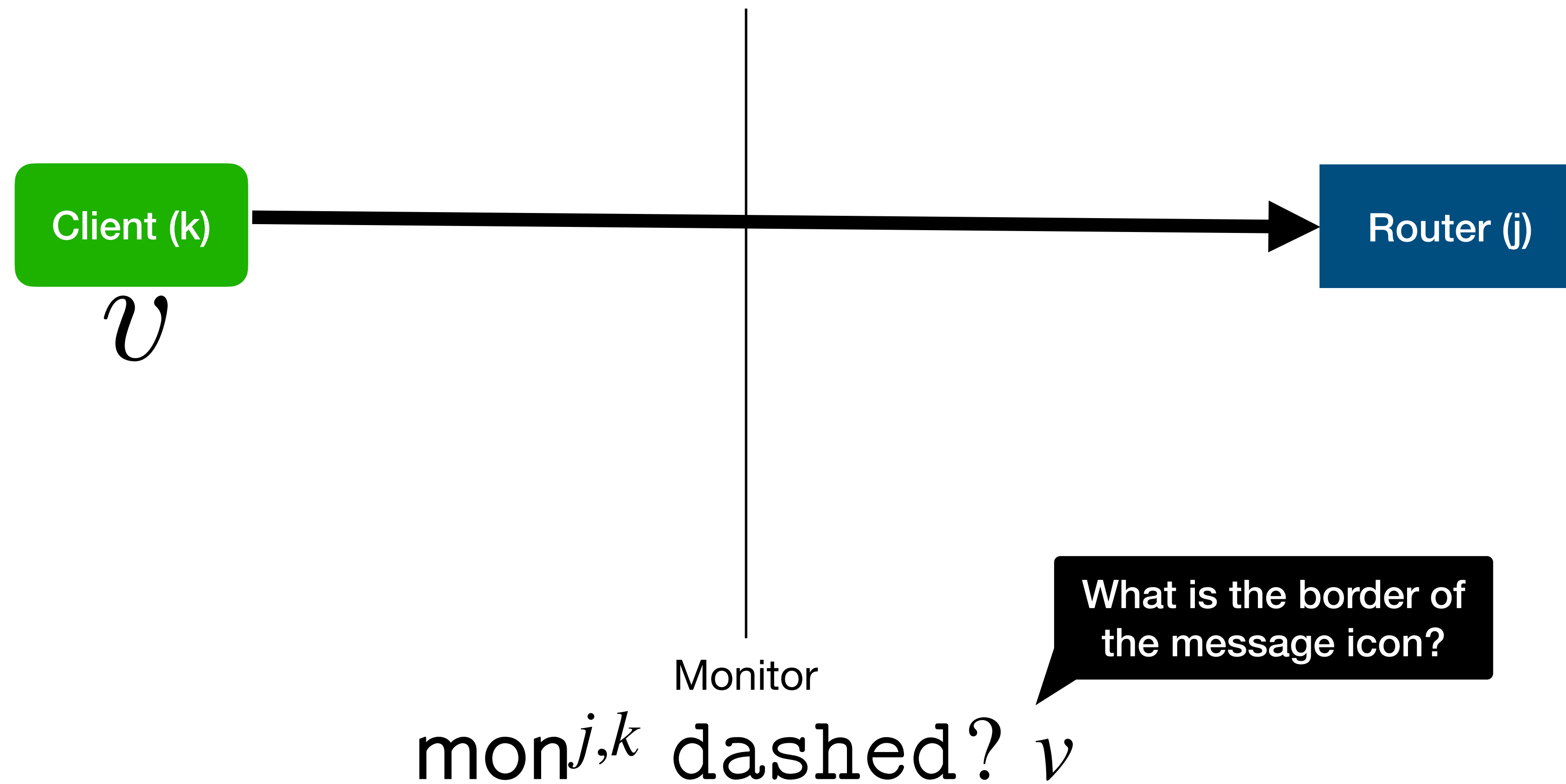
Blame Assignment

“Classic” Sequential Blame Assignment



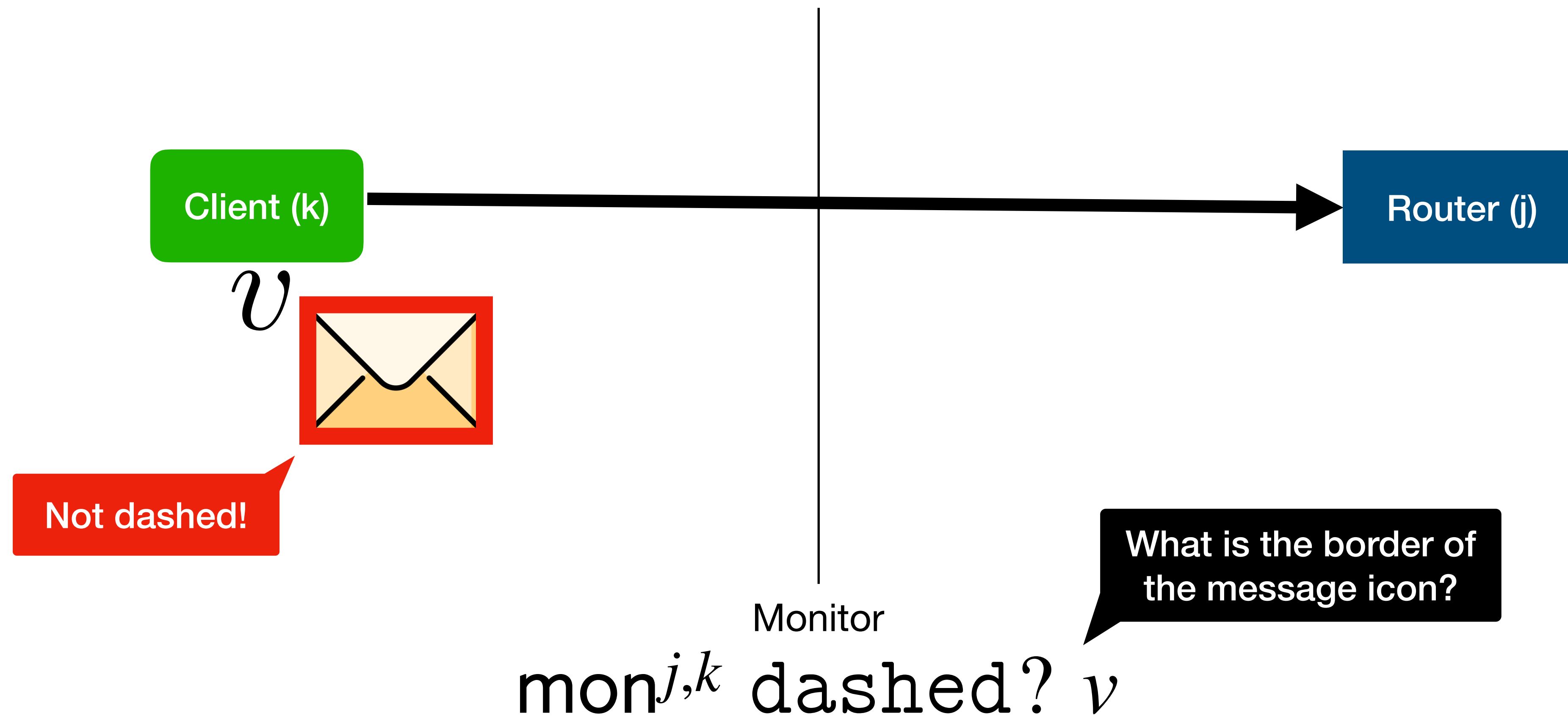
Blame Assignment

“Classic” Sequential Blame Assignment



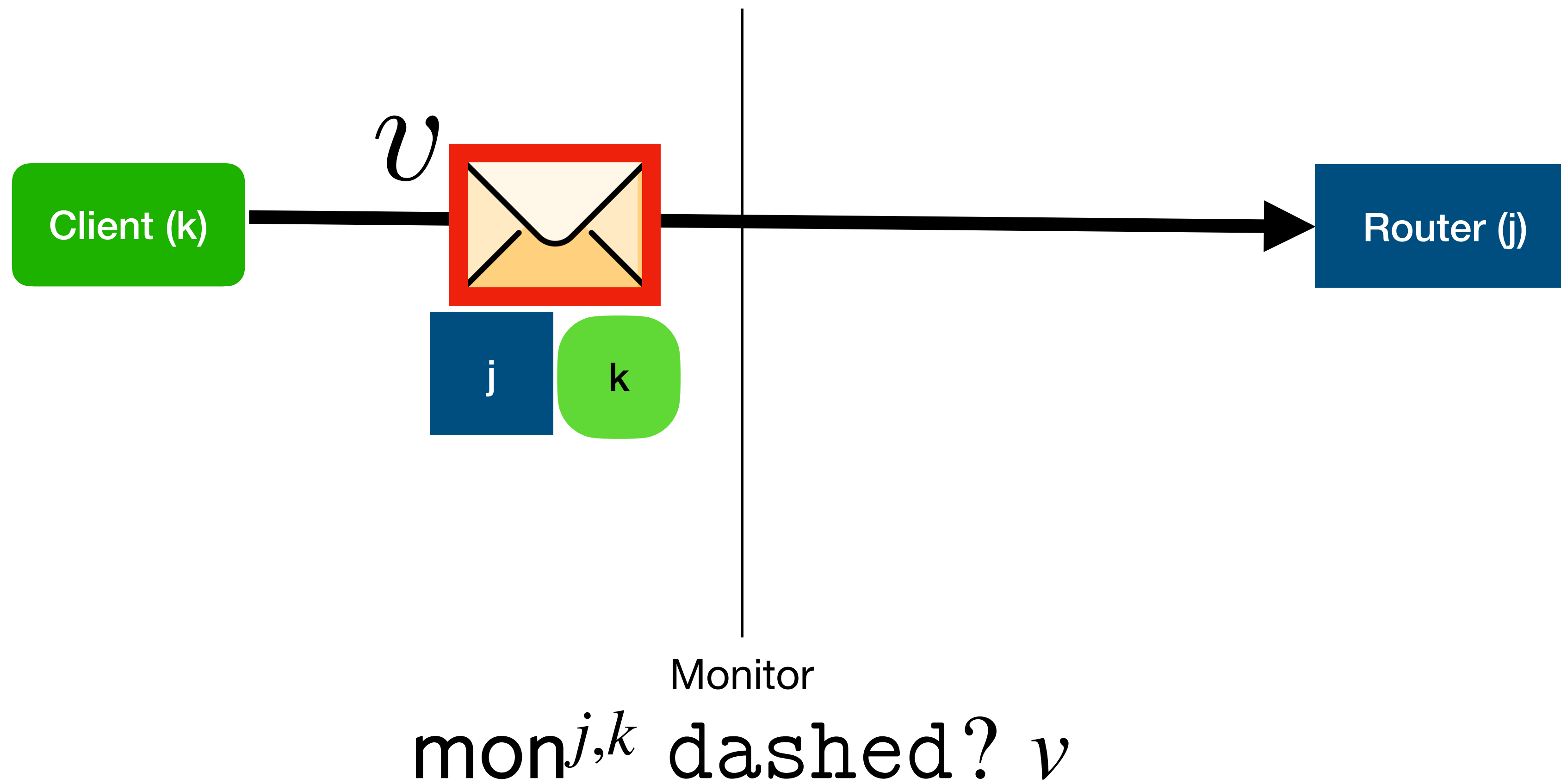
Blame Assignment

“Classic” Sequential Blame Assignment



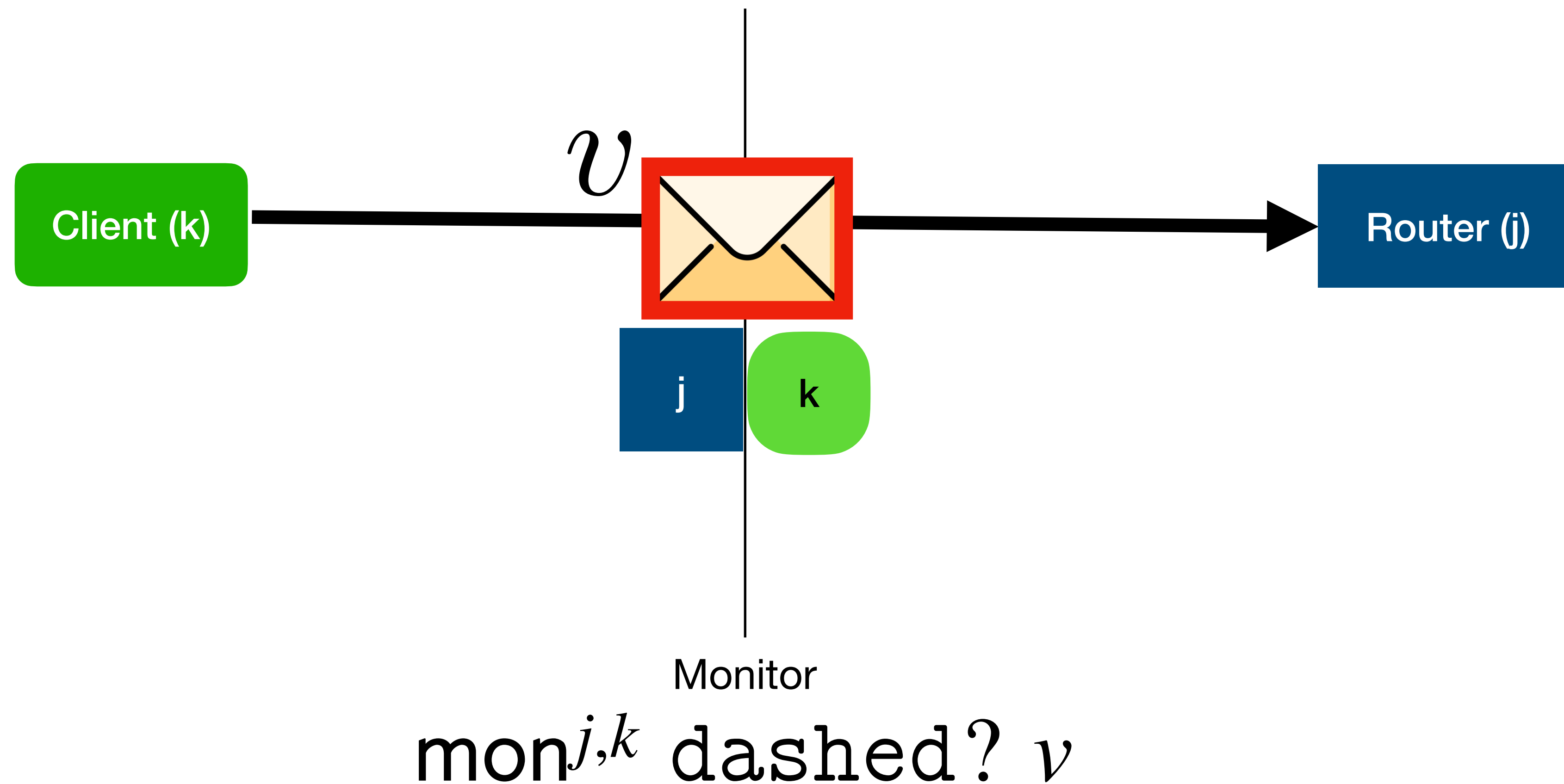
Blame Assignment

“Classic” Sequential Blame Assignment



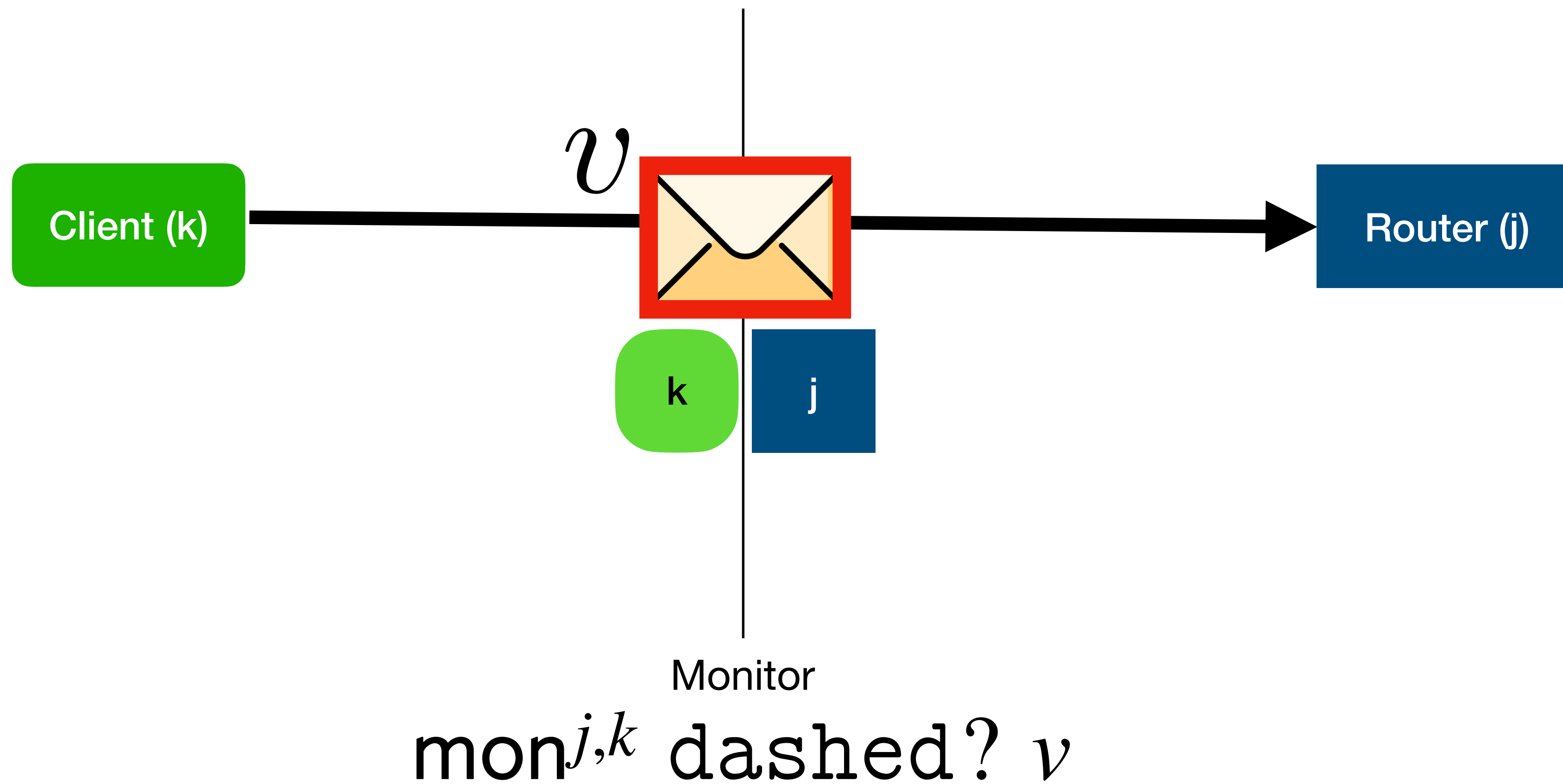
Blame Assignment

“Classic” Sequential Blame Assignment



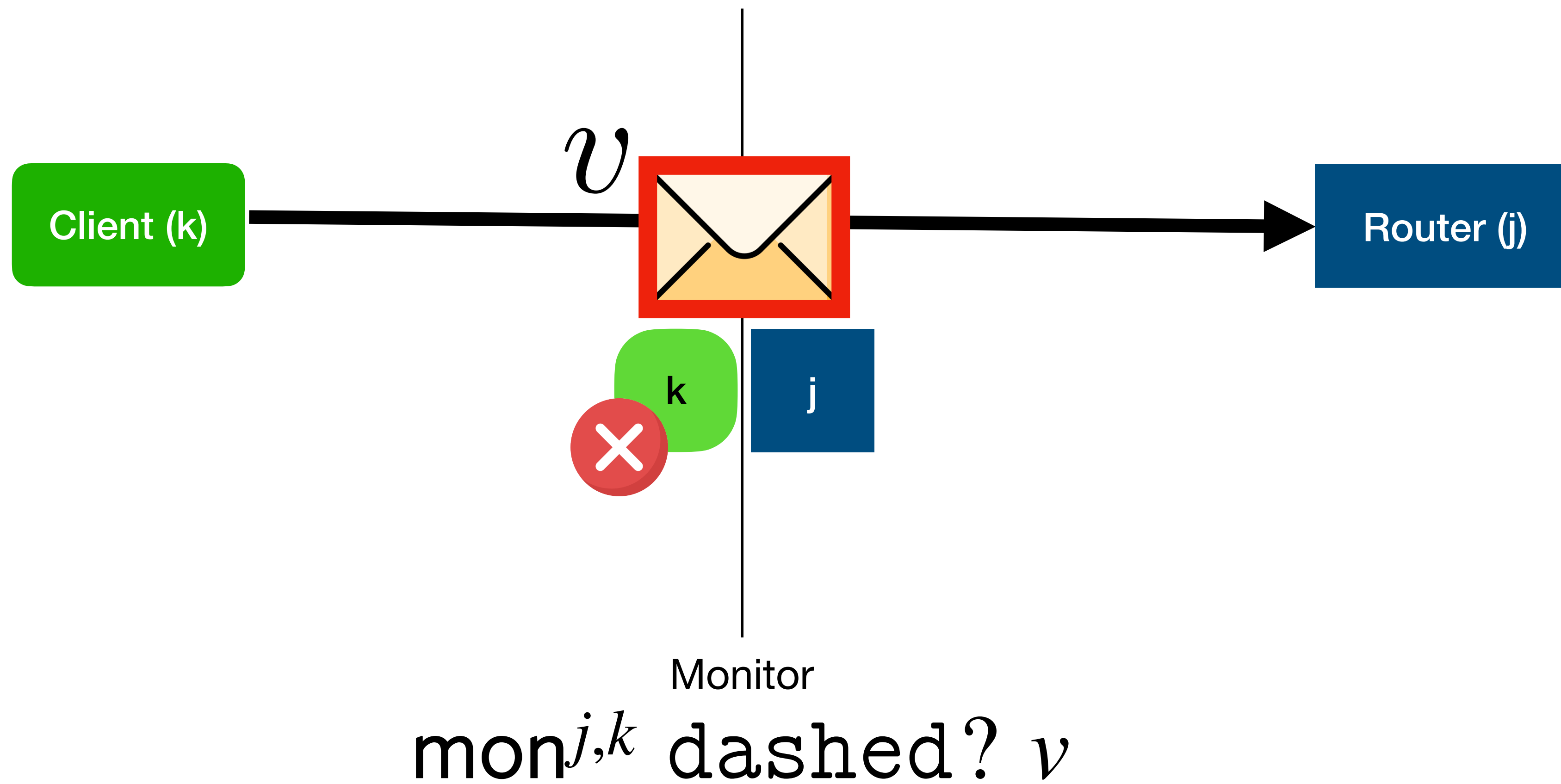
Blame Assignment

“Classic” Sequential Blame Assignment



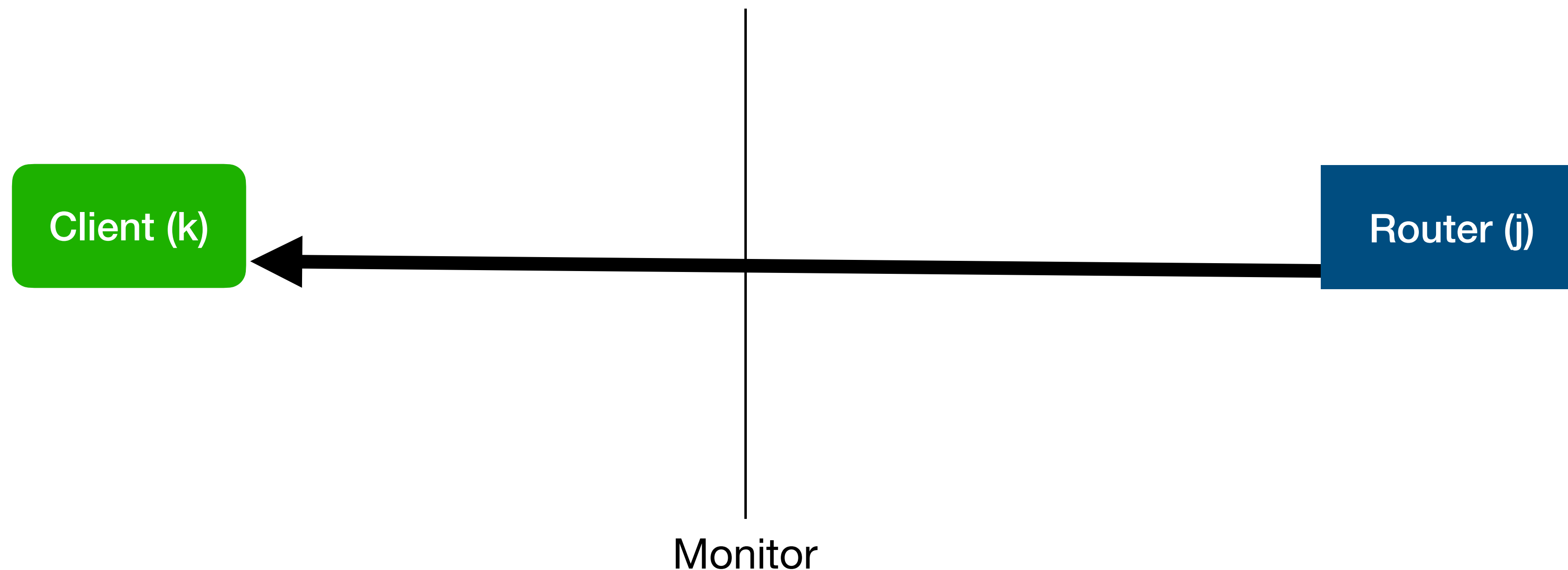
Blame Assignment

“Classic” Sequential Blame Assignment



Blame Assignment

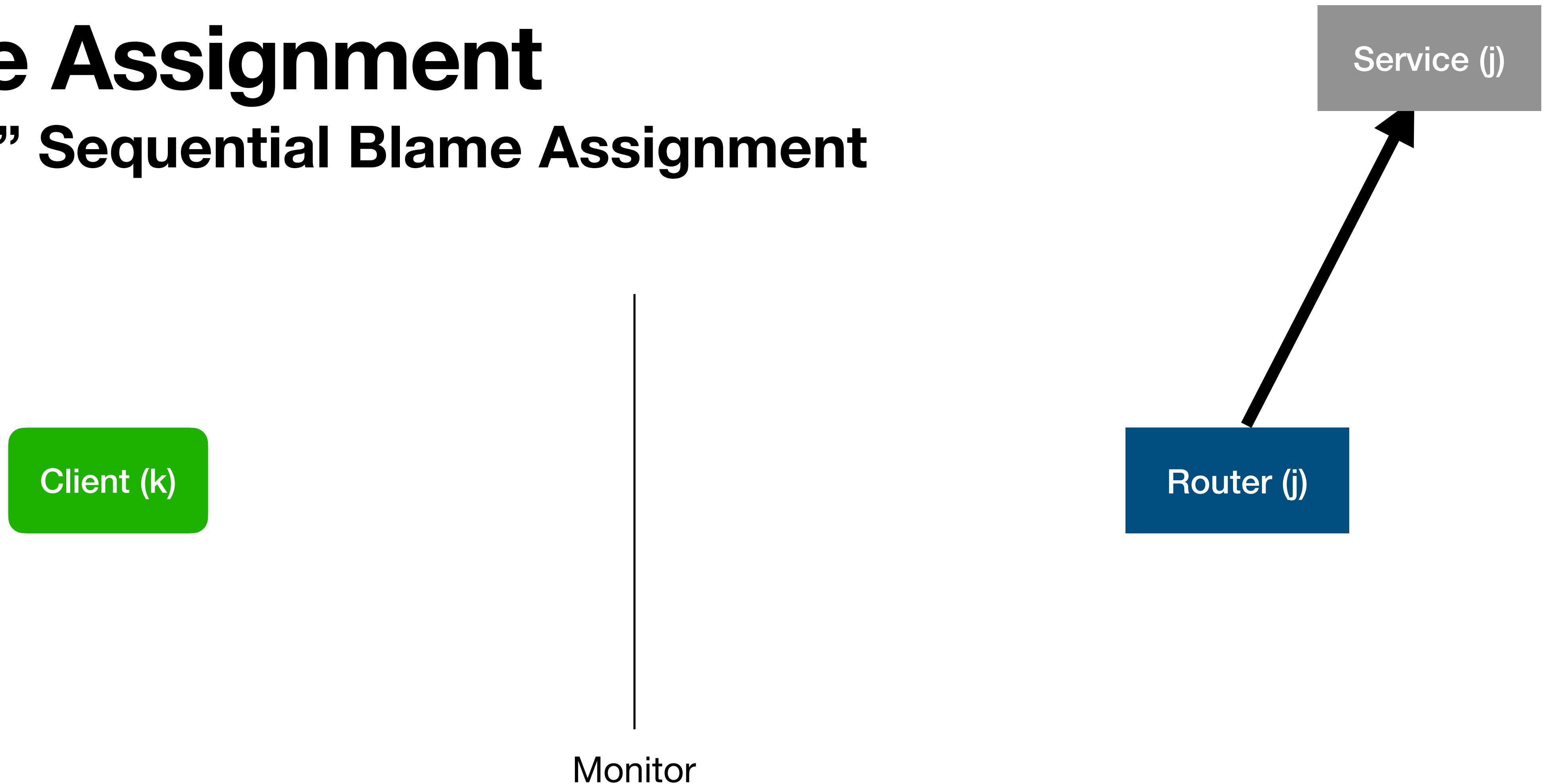
“Classic” Sequential Blame Assignment



No “return value” in an actor system

Blame Assignment

“Classic” Sequential Blame Assignment



Sequential contract systems **ignore communication with other parties**

Blame Assignment

Router is responsible for forwarding the message to the correct service

```
(message/c request
  dashed?
  (specific-recipient service)
  (ensures/c
    (message/c reply solid?
      (specific-recipient (first payload))
      unconstrained/c))))
```

Client (k)

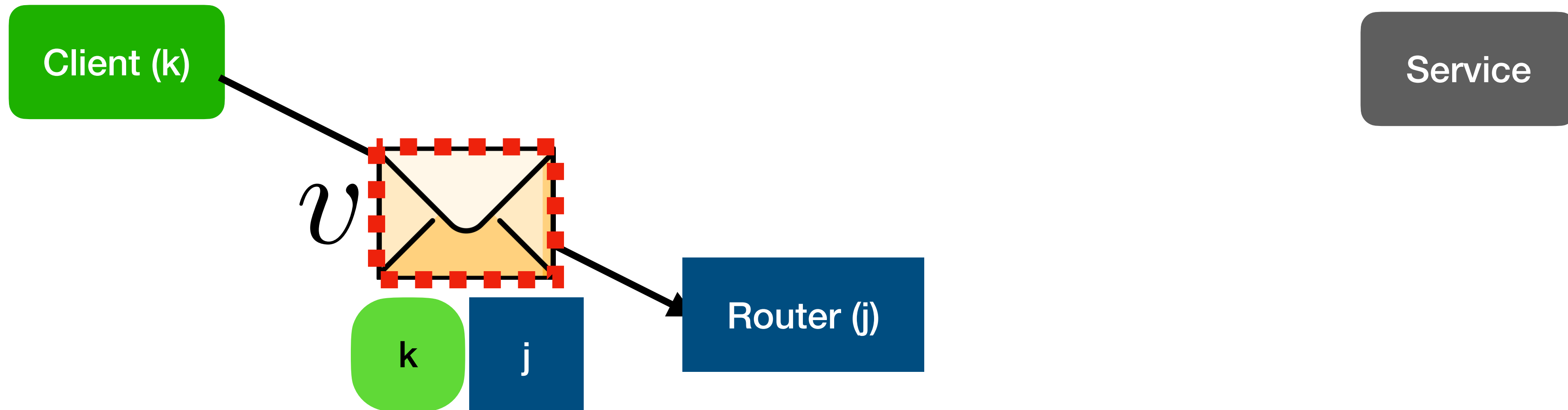
Service

Router (j)

Blame Assignment

Router is responsible for forwarding the message to the correct service

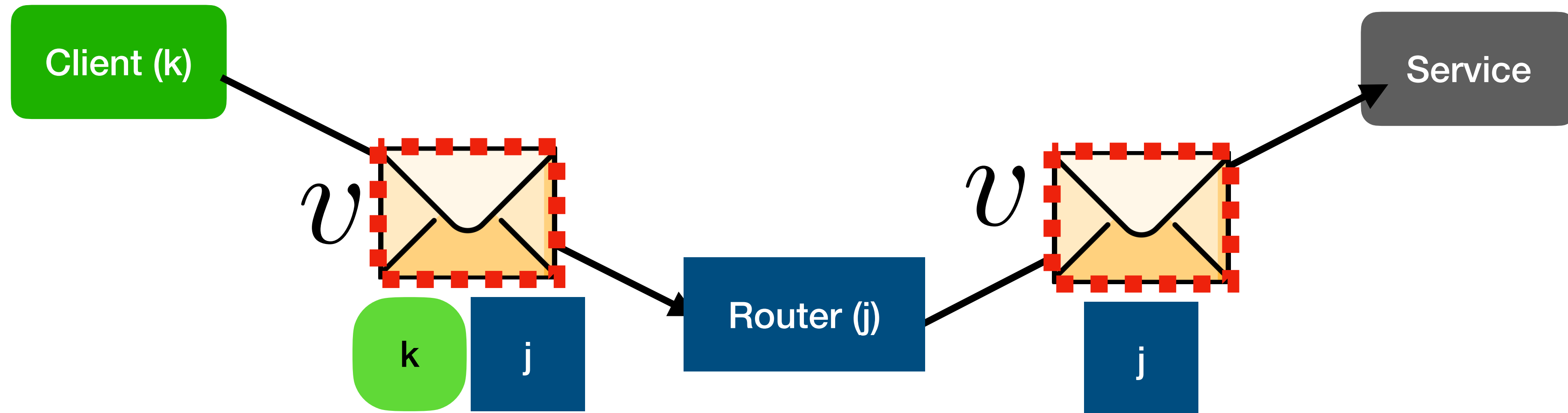
```
(message/c request
dashed?
(specific-recipient service)
(ensures/c
  (message/c reply solid?
    (specific-recipient (first payload))
    unconstrained/c))))
```



Blame Assignment

Router is responsible for forwarding the message to the correct service

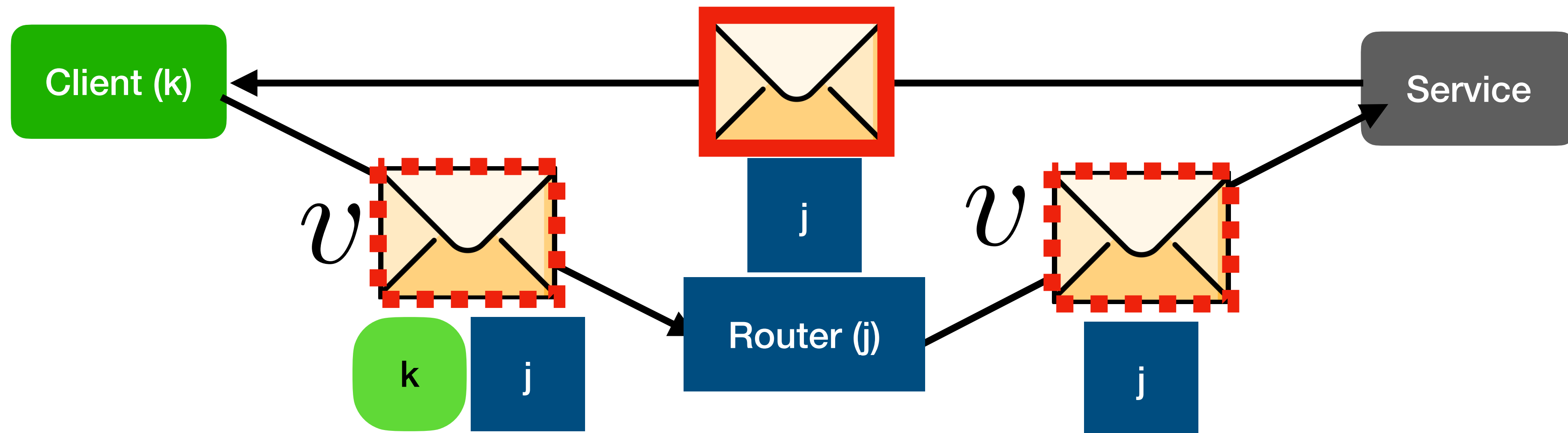
```
(message/c request  
  dashed?  
    (specific-recipient service)  
    (ensures/c  
      (message/c reply solid?  
        (specific-recipient (first payload))  
        unconstrained/c))))))
```



Blame Assignment

Router is responsible for forwarding the message to the correct service

```
(message/c request
dashed?
(specific-recipient service)
(ensures/c
  (message/c reply solid?
    (specific-recipient (first payload))
    unconstrained/c))))
```



Validation

Expressiveness

Contracts for reactive design patterns

 [README.md](#)

Blame-correct Support for Receiver Properties in Recursively-Structured Actor Contracts (Artifact)

This replication package contains an executable semantics as a PLT-Redex implementation for the formalisation presented in the paper. This executable semantics is packaged inside the `redex` folder. The replication package also contains an implementation of our contract language in Racket, which is located inside the `racket-implementation` folder.

Goal

The goal of our replication package is to provide an executable semantics to largely corresponds to the formal semantics in the paper. This validates the correctness of our formal description as it is mostly derived from this executable semantics.

Moreover, the replication package contains a Racket implementation of our contract system which contains the examples presented in the `Communication contracts in practice section` of the paper.

Prerequisites

To run our replication package the following packages are required:

Racket implementation available

• racket (version 8.2 or higher)
• a Debian 11 virtual machine
• a Docker virtual machine for



Validation

Expressiveness

Contracts for reactive design patterns

[README.md](#)

Blame-correct Support for Receiver Properties in Recursively-Structured Actor Contracts (Artifact)

This replication package contains an executable semantics as a PLT-Redex implementation for the formalisation presented in the paper. This executable semantics is packaged inside the `redex` folder. The replication package also contains an implementation of our contract language in Racket, which is located inside the `racket-implementation` folder.

Goal

The goal of our replication package is to provide an executable semantics to largely corresponds to the formal semantics in the paper. This validates the correctness of our formal description as it is mostly derived from this executable semantics.

Moreover, the replication package contains a Racket implementation of our contract system which contains the examples presented in the `Communication contracts in practice` section of the paper.

Prerequisites

To run our replication package the following packages are required:

Racket implementation available



Formal

Proved Blame Correctness

$$\begin{aligned}
 e &::= \dots \mid \text{mon}_l^{j,k} e e & v &::= \dots \mid \kappa \mid \kappa r \mid \kappa c \mid \text{mon}_l^{j,k} v v \\
 \kappa &::= \kappa \rightarrow \lambda x. e \mid \text{flat}(e) \mid \text{behavior}/c (\kappa m \vee \dots \vee \kappa m) \\
 \kappa m &::= \text{message}/c \tau (\lambda x. e) \kappa (\lambda x. e) \mid \emptyset & \kappa r &::= \text{receiver}(e) \\
 \kappa c &::= \kappa m \mid \text{only}/c (\kappa m, \dots) \mid \text{ensures}/c (\kappa m, \dots) & j, k, l &\in \text{BlameLabel}
 \end{aligned}$$

Value-based contracts on the payload

$$\begin{aligned}
 \text{stack}_p (\text{mon}_l^{j,k} \kappa_1 v_1) v_2 &= (\text{stack}_p v_1 (\text{mon}_l^{k,j} \kappa_p v_2)) \\
 \text{where } \kappa_1 &= \text{behavior}/c \kappa m_1 \vee \dots \vee \kappa m_n & \text{message}/c \tau &\xrightarrow{E} \langle \pi, \text{monitored } e_\kappa ([x \mapsto v] e), t, M \rangle \\
 \text{stack}_p \pi v_2 &= v_2 & [\text{M-CONGRUENCE}] &\hat{e} \rightarrow \hat{e}' \Rightarrow \langle \pi, \text{monitored } e_\kappa E[\hat{e}], t, M \rangle \rightarrow_\alpha \langle \pi, \text{monitored } e_\kappa E[\hat{e}'], t, M \rangle
 \end{aligned}$$

Receiver contracts

$$\begin{aligned}
 \text{stack}'_r (\text{mon}_l^{j,k} \kappa_1 v_1) c &= \text{stack}'_r v_1 (\lambda x. \lambda y. (\text{check}^k (\kappa r x) (c x y))) \\
 \text{stack}'_r \pi c &= c \pi & [\text{M-SEND}] &\text{send } v_1 \tau v_2 \rightarrow \overline{\text{send}} e'_\kappa \tau v_4 \Rightarrow \langle \pi, \text{monitored } e_\kappa E[\text{send } v_1 \tau v_2], t, M \rangle \\
 \text{where } \kappa_1 &= \text{behavior}/c \kappa m_1 \vee \dots \vee \kappa m_n & \text{message}/c \tau &\xrightarrow{E} \langle \pi, \text{monitored } e_\kappa E[\text{send } \kappa c' \pi' \tau v_2], t_1, M_1 \rangle, \langle \pi', e, t_2, M_2 \rangle \} \uplus \mathcal{A} \\
 \text{stack}_r v &= \text{stack}'_r v (\lambda x. x) & \rightarrow_\alpha &\langle \pi, \text{monitored } e_\kappa E[\text{nil}], (\tau, v_2) \cdot t_1, M_1 \rangle, \langle \pi', e_2, t_2, (\tau, \kappa c', v) \cdot M \rangle \} \uplus \mathcal{A}
 \end{aligned}$$

Communication contracts

$$\begin{aligned}
 \text{stack}_c (\text{mon}_l^{j,k} \kappa_1 v_1) &= (\lambda x. \text{mon}_c^j \kappa c^j x) \circ \text{stack}_c v_1 \\
 \text{where } \kappa_1 &= \text{behavior}/c \kappa m_1 \vee \dots \vee \kappa m_n & \text{message}/c \tau &_ _ \kappa c \in \{\kappa m_1, \dots, \kappa m_n\} \\
 \text{stack}_c \pi &= \emptyset
 \end{aligned}$$

Correct Blame Plan



Dimoulas, C., Findler, R. B., Flanagan, C., & Felleisen, M. (2011, January). Correct blame for contracts: no more scapegoating. *POPL*



A party can only be blamed if the ***value*** causing the contract violation is owned by that party (*cf. Dimoulas et al.*)

Correct Blame Plan



Dimoulas, C., Findler, R. B., Flanagan, C., & Felleisen, M. (2011, January). Correct blame for contracts: no more scapegoating. *POPL*



A party can only be blamed if the **value** causing the contract violation is owned by that party (*cf. Dimoulas et al.*)

Goal: annotate the source program with **ownership annotations** and propagate through program execution

Correct Blame Plan



A party can only be blamed if the **value** causing the contract violation is owned by that party (*cf. Dimoulas et al.*)

Goal: annotate the source program with **ownership annotations** and propagate through program execution



At blame errors check if blame label aligns with ownership annotation

Correct Blame Plan

 Dimoulas, C., Findler, R. B., Flanagan, C., & Felleisen, M. (2011, January). Correct blame for contracts: no more scapegoating. POPL



A party can only be blamed if the **value** causing the contract violation is owned by that party (*cf. Dimoulas et al.*)

Goal: annotate the source program with **ownership annotations** and propagate through program execution



At blame errors check if blame label aligns with ownership annotation

Main challenge: relate communication behavior to value ownership

Conclusion



Expectations as Contract Specifications

- Expected payload**
Payload should contain actor reference
- Expected actor interface**
The router expects a **request** message from the client
- Expected communication behavior**
Router forwards message to the service and service replies to the client

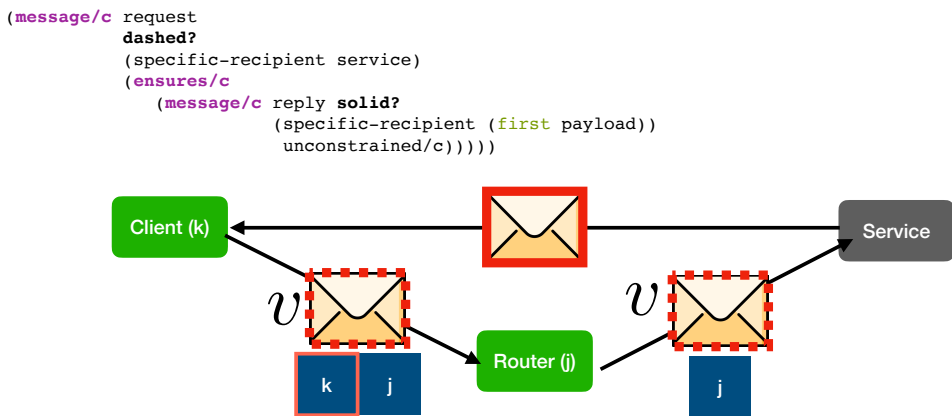
Message contracts

Actor interface contracts

Communication contracts
Receiver contracts

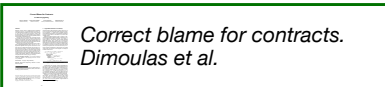
Blame Assignment

Server is responsible for forwarding the message to the correct service

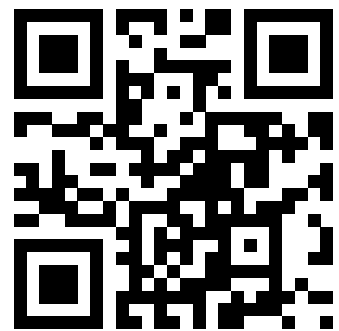


Correct Blame Assignment Plan

- A party can only be blamed if the **value** causing the contract violation is owned by that party (cf. Dimoulas et al.)
- Goal:** annotate the source program with **ownership annotations** and propagate through program execution
- At blame errors check if blame label aligns with ownership annotation



Artifact containing executable semantics
and Racket implementation
<https://vdb.space/artifacts/icfp24>



Full paper
<https://vdb.space/papers/icfp2024.pdf>

How to reach me:

- bram.vandenbogaerde@vub.be
- [@bramvdbogaerde](https://twitter.com/bramvdbogaerde)